

M2 IABD — DEEP REINFORCEMENT LEARNING

Onze algorithmes, quatre environnements

Line World • Grid World • TicTacToe versus Random • Bobail

Omar Kaabi • Awatef B. • Viet Hoang

[compléter les noms complets]

github.com/KaabiOmar/drl-project

But de l'exercice

- Tester différents algorithmes d'apprentissage par renforcement sur différents environnements.
- Comprendre en quoi chacun est utile, et dans quels cas l'utiliser.

La question n'est pas « lequel est le meilleur » mais « lequel, pour quelle situation »

- **Ce que nous avons produit** : 165 entraînements, plus de 55 millions d'épisodes joués, 11 agents $\times$ 4 environnements $\times$ 3 graines, 0 échec.
- **Ce que nous mesurons** : score de la politique obtenue, longueur des parties, temps par coup — à 1 000, 10 000, 100 000 et 1 000 000 d'épisodes.

Line World

IMPLÉMENTATION

Un couloir de 5 cases. Le pion démarre au centre. Atteindre l'extrémité droite rapporte +1, l'extrémité gauche -1. Les deux bouts sont terminaux.

SON RÔLE DANS L'ÉTUDE

Le cas le plus simple possible. Il sert de test de bon fonctionnement : un agent correct doit y atteindre +1.

CE QU'IL REÇOIT EN ENTRÉE

- vecteur de 5 valeurs : one-hot de la position
- masque d'actions : toujours [1, 1] — les murs sont absorbants

CE QU'IL RENVOIE EN SORTIE

- 2 actions : 0 = gauche, 1 = droite
- récompense +1 / -1 en bout de couloir, 0 sinon
- environ 4 pas par partie

Grid World

IMPLÉMENTATION

Une grille 5×5. Départ en haut à gauche. Le coin bas-droite est un but (+1), le coin haut-droite un piège (-1). Les murs sont absorbants.

SON RÔLE DANS L'ÉTUDE

Introduit un horizon long : la récompense est loin du départ, ce qui teste la propagation du crédit.

CE QU'IL REÇOIT EN ENTRÉE

- vecteur de 25 valeurs : one-hot de la case
- masque d'actions : toujours tout à 1

CE QU'IL RENVOIE EN SORTIE

- 4 actions : haut, bas, gauche, droite
- récompense +1 au but, -1 au piège, 0 sinon
- environ 53 pas par partie — dix fois plus que les autres

TicTacToe versus Random

IMPLÉMENTATION

Morpion 3×3. L'agent joue les croix et commence. L'adversaire joue au hasard parmi les cases libres, à l'intérieur de `step()` : vu de l'agent, l'environnement reste un problème à un seul joueur.

SON RÔLE DANS L'ÉTUDE

Premier environnement à deux joueurs, et le premier où les algorithmes se différencient réellement.

CE QU'IL REÇOIT EN ENTRÉE

- vecteur de 27 valeurs : 3 plans de 9 cases — vide / agent / adversaire
- masque d'actions : 1 sur les cases libres

CE QU'IL RENVOIE EN SORTIE

- 9 actions : la case jouée
- récompense +1 victoire, -1 défaite, 0 match nul
- environ 4 coups par partie

Bobail

IMPLÉMENTATION

Plateau 5×5, 5 pions par joueur, une boule neutre au centre.
Un tour = déplacer la boule d'une case, puis faire glisser un pion en ligne droite jusqu'à l'obstacle. On gagne en ramenant la boule sur sa propre ligne, ou en l'enfermant.

SON RÔLE DANS L'ÉTUDE

Notre environnement libre, et le plus riche : espace d'actions large, tour en deux phases, deux niveaux d'adversaire.

CE QU'IL REÇOIT EN ENTRÉE

- vecteur de 77 valeurs : 3 plans de 25 cases + la phase du tour en one-hot
- masque d'actions : moins de 5 % des 208 actions sont légales à un instant donné

CE QU'IL RENVOIE EN SORTIE

- 208 actions : 8 directions pour la boule, puis case de départ × direction pour un pion
- récompense +1 victoire, -1 défaite
- adversaire aléatoire ou heuristique, au choix

Les onze agents, en trois familles

Famille	Agents	Ce que chacun ajoute au précédent
Valeur	dqn → ddqn → ddqn_er → ddqn_per	réseau cible, puis mémoire de rejeu, puis priorisation
Gradient de politique	reinforce → baseline → critique	retours centrés, puis baseline apprise $V(s)$
	ppo	rapport clippé + $GAE(\lambda)$: plusieurs passes par lot
Planification	random_rollout, mcts_uct	aucun apprentissage : simulation à chaque coup
Référence	random	uniforme sur les coups légaux

— **Un delta par agent.** À l'intérieur d'une famille, chacun n'ajoute qu'un mécanisme : un effet observé s'attribue donc à une cause précise.

— **Architecture identique partout** — deux couches de 128 unités, $\gamma = 0,99$. C'est ce qui rend la comparaison honnête.

Comment nous mesurons

- **La politique obtenue, jamais celle d'entraînement.** $\varepsilon = 0$, argmax au lieu d'échantillonnage, sur 200 parties. Un score mesuré avec 10 % d'exploration ne mesure rien.
- **Trois graines par configuration.** L'écart entre graines est souvent plus parlant que le score : il distingue un algorithme fiable d'un algorithme chanceux.
- **Une graine différente par partie d'évaluation,** décalée loin des graines d'entraînement : on mesure sur des parties non vues, tout en restant reproductible.

Score : +1 victoire • -1 défaite • 0 nul ou limite de pas atteinte

ENVIRONNEMENT 1 / 4

Line World

Démonstration

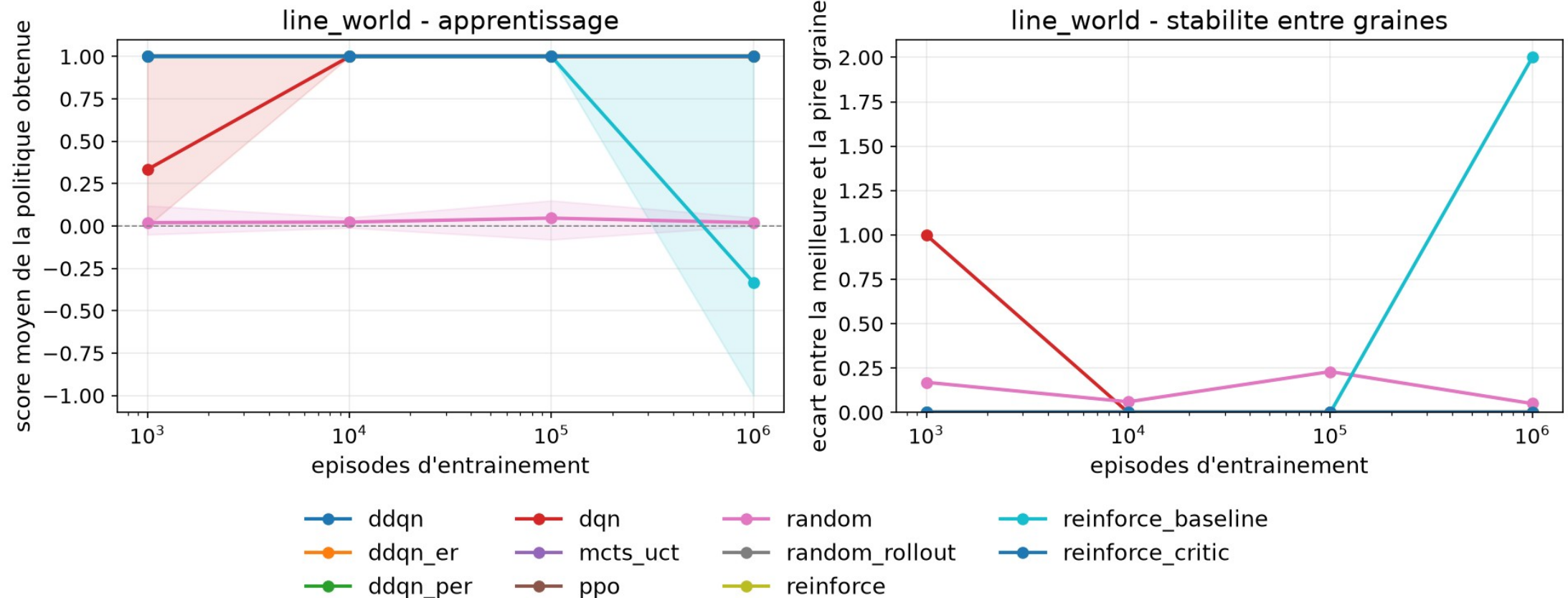
```
python -m src.gui.app --env line_world --agent dqn
```

Récapitulatif des agents et résultats

Agent	Objectif et particularité	Score final
Random	joue au hasard parmi les coups légaux — la barre à battre	+0,02
Deep Q-Learning	apprend une valeur par action, joue l'argmax	+1,00
Double Deep Q	ajoute un réseau cible : la cible arrête de bouger	+1,00
Double DQN + Replay	rejoue ses anciennes parties : casse la corrélation	+1,00
Double DQN + Replay priorisé	rejoue en priorité ce qu'il rate encore	+1,00
REINFORCE	apprend la politique directement, en fin de partie	+1,00
REINFORCE + baseline	centre les retours : moins de variance	-0,33
REINFORCE + critique	remplace la baseline par un réseau $V(s)$	+1,00
PPO (A2C style)	borne chaque mise à jour : plusieurs passes sans exploser	+1,00
Random Rollout	simule 20 fins de partie par coup, sans apprendre	+1,00
MCTS (UCT)	construit un arbre de 200 simulations par coup	+1,00

Mesuré jusqu'à 1 000 000 d'épisodes. Score de la politique obtenue, moyenne sur 3 graines. Hyperparamètres communs : deux couches de 128 unités, $\gamma = 0,99$.

Courbes d'apprentissage



À gauche le score au fil de l'entraînement, à droite l'écart entre graines : plus il est bas, plus l'algorithme est fiable.

Ce que cet environnement nous apprend

- **Environnement saturé.** Dix agents sur onze atteignent +1,00 dès 1 000 épisodes. Le tableau ne compare rien : la tâche est trop facile.
- **Une anomalie instructive.** REINFORCE + baseline tient +1,00 jusqu'à 100 000 épisodes puis chute à -0,33 à un million — deux graines sur trois divergent.
- **Ce que ça valide quand même.** Le protocole détecte bien un apprentissage : l'agent aléatoire reste à +0,02.

COMMENT L'AMÉLIORER

Allonger le couloir, rendre les transitions stochastiques, ou éloigner la récompense — pour que la tâche redevienne discriminante.

ENVIRONNEMENT 2 / 4

Grid World

Démonstration

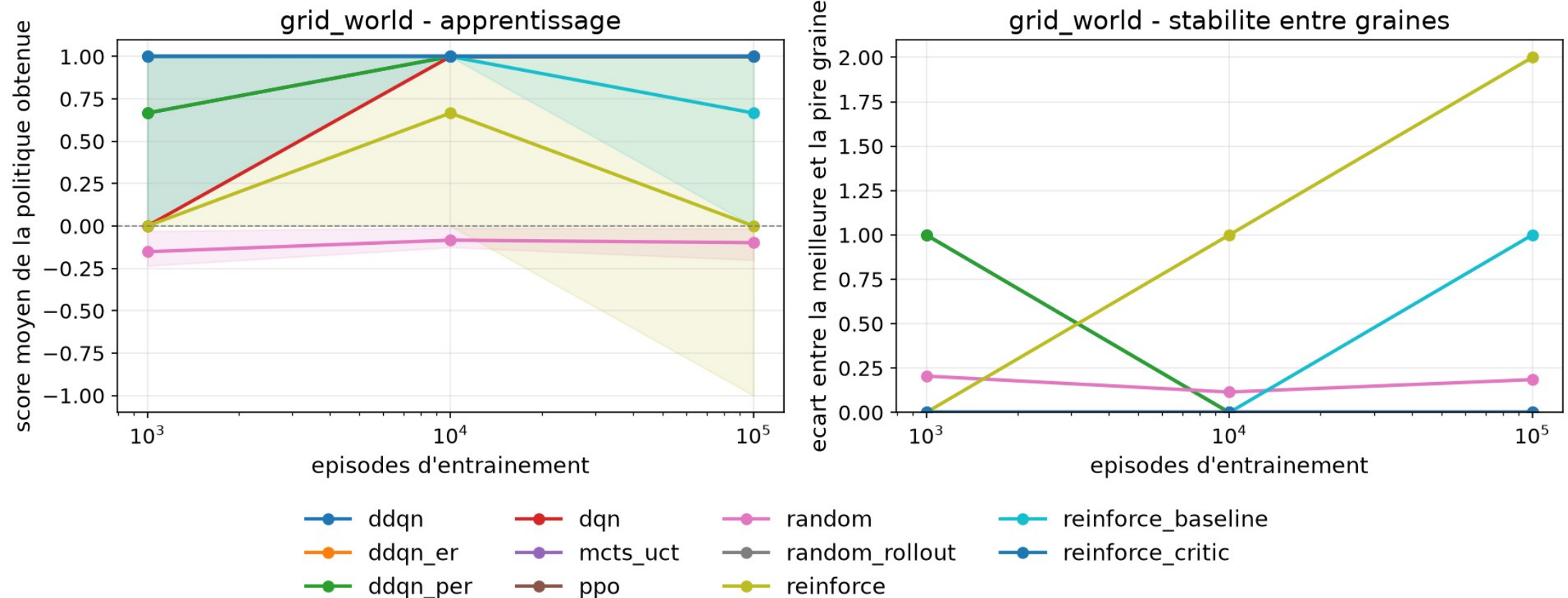
```
python -m src.gui.app --env grid_world --agent ppo
```

Récapitulatif des agents et résultats

Agent	Objectif et particularité	Score final
Random	joue au hasard parmi les coups légaux — la barre à battre	-0,10
Deep Q-Learning	apprend une valeur par action, joue l'argmax	+1,00
Double Deep Q	ajoute un réseau cible : la cible arrête de bouger	+1,00
Double DQN + Replay	rejoue ses anciennes parties : casse la corrélation	+1,00
Double DQN + Replay priorisé	rejoue en priorité ce qu'il rate encore	+1,00
REINFORCE	apprend la politique directement, en fin de partie	+0,00
REINFORCE + baseline	centre les retours : moins de variance	+0,67
REINFORCE + critique	remplace la baseline par un réseau $V(s)$	+1,00
PPO (A2C style)	borne chaque mise à jour : plusieurs passes sans exploser	+1,00
Random Rollout	simule 20 fins de partie par coup, sans apprendre	+1,00
MCTS (UCT)	construit un arbre de 200 simulations par coup	+1,00

Mesuré jusqu'à 100 000 épisodes — le palier 1 M coûterait 38,8 h. Score de la politique obtenue, moyenne sur 3 graines. Hyperparamètres communs : deux couches de 128 unités, $\gamma = 0,99$.

Courbes d'apprentissage



À gauche le score au fil de l'entraînement, à droite l'écart entre graines : plus il est bas, plus l'algorithme est fiable.

Ce que cet environnement nous apprend

- **Saturé à partir de 10 000 épisodes**, mais plus lent à converger que Line World : l'horizon est plus long, 53 pas par partie contre 4.
- **REINFORCE ne se stabilise pas**. Il oscille entre 0,00 et 0,67 quand sa variante à critique atteint et conserve +1,00 : la baseline apprise règle un problème que la baseline constante ne règle pas.
- **Le coût domine tout**. 4 430 parties/s contre 63 000 sur Line World. Cet environnement consomme à lui seul 3,9 h de nos 6,4 h de campagne.

COMMENT L'AMÉLIORER

Ajouter des obstacles et des pièges intermédiaires, agrandir la grille, et porter le palier d'un million sur une machine plus rapide.

ENVIRONNEMENT 3 / 4

TicTacToe versus Random

Démonstration

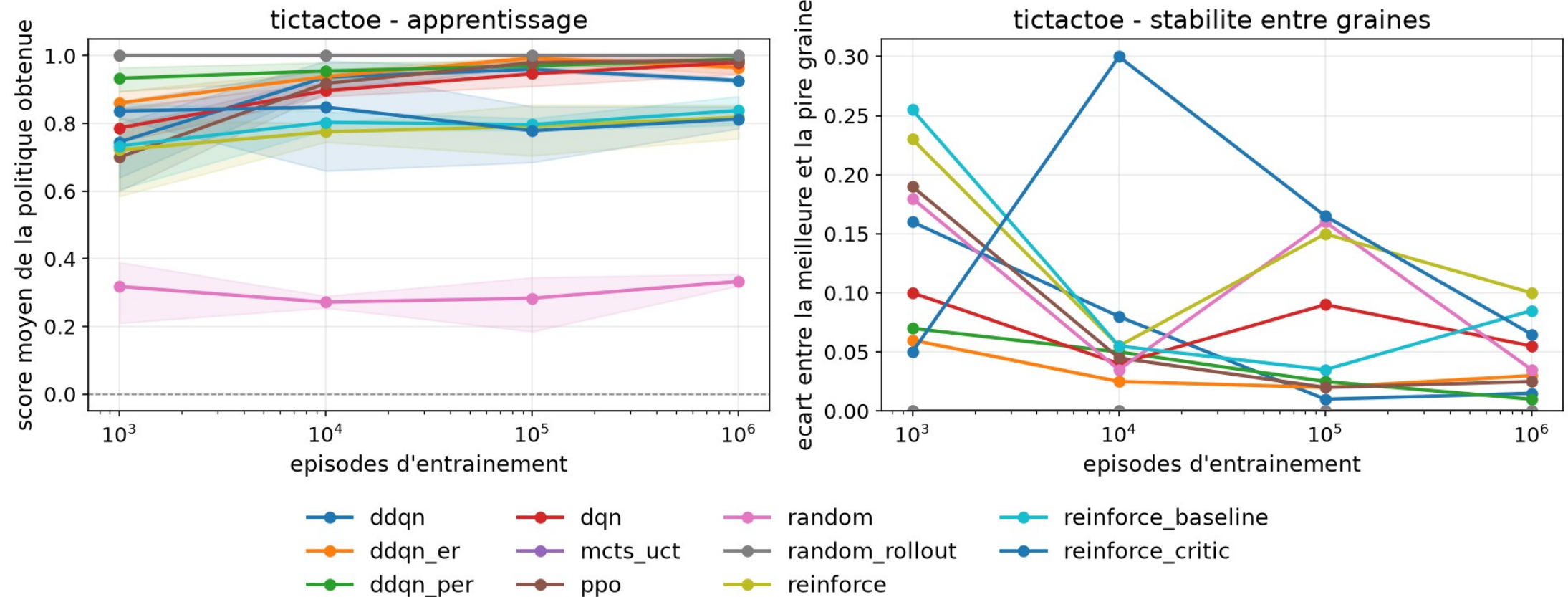
```
python -m src.gui.app --env tictactoe --agent human
```

Récapitulatif des agents et résultats

Agent	Objectif et particularité	Score final
Random	joue au hasard parmi les coups légaux — la barre à battre	+0,33
Deep Q-Learning	apprend une valeur par action, joue l'argmax	+0,98
Double Deep Q	ajoute un réseau cible : la cible arrête de bouger	+0,93
Double DQN + Replay	rejoue ses anciennes parties : casse la corrélation	+0,96
Double DQN + Replay priorisé	rejoue en priorité ce qu'il rate encore	+0,99
REINFORCE	apprend la politique directement, en fin de partie	+0,82
REINFORCE + baseline	centre les retours : moins de variance	+0,84
REINFORCE + critique	remplace la baseline par un réseau $V(s)$	+0,81
PPO (A2C style)	borne chaque mise à jour : plusieurs passes sans exploser	+0,98
Random Rollout	simule 20 fins de partie par coup, sans apprendre	+1,00
MCTS (UCT)	construit un arbre de 200 simulations par coup	+1,00

Mesuré jusqu'à 1 000 000 d'épisodes. Score de la politique obtenue, moyenne sur 3 graines. Hyperparamètres communs : deux couches de 128 unités, $\gamma = 0,99$.

Courbes d'apprentissage



À gauche le score au fil de l'entraînement, à droite l'écart entre graines : plus il est bas, plus l'algorithme est fiable.

Ce que cet environnement nous apprend

- **Le premier environnement qui discrimine.** Les agents s'étalent de 0,82 à 1,00, contre 0,33 pour l'agent aléatoire.
- **La famille REINFORCE plafonne à 0,82** et ne progresse plus après 10 000 épisodes. PPO, même famille, atteint 0,98 : le rapport clippé est le mécanisme décisif.
- **Le classement final n'est pas un résultat.** Famille valeur et PPO sont entre 0,93 et 0,99, avec des écarts entre graines du même ordre. Ce qui les sépare est la stabilité : à 10 000 épisodes, 0,156 d'écart pour DQN contre 0,007 pour Double DQN.

COMMENT L'AMÉLIORER

Remplacer l'adversaire aléatoire par un adversaire compétent. Notre meilleur agent gagne 99,3 % des parties mais ignore 31,8 % des menaces adverses : il a appris à exploiter un adversaire faible, pas à jouer au morpion.

ENVIRONNEMENT 4 / 4

Bobail

Démonstration

```
python -m src.gui.app --env bobail --agent ppo
```

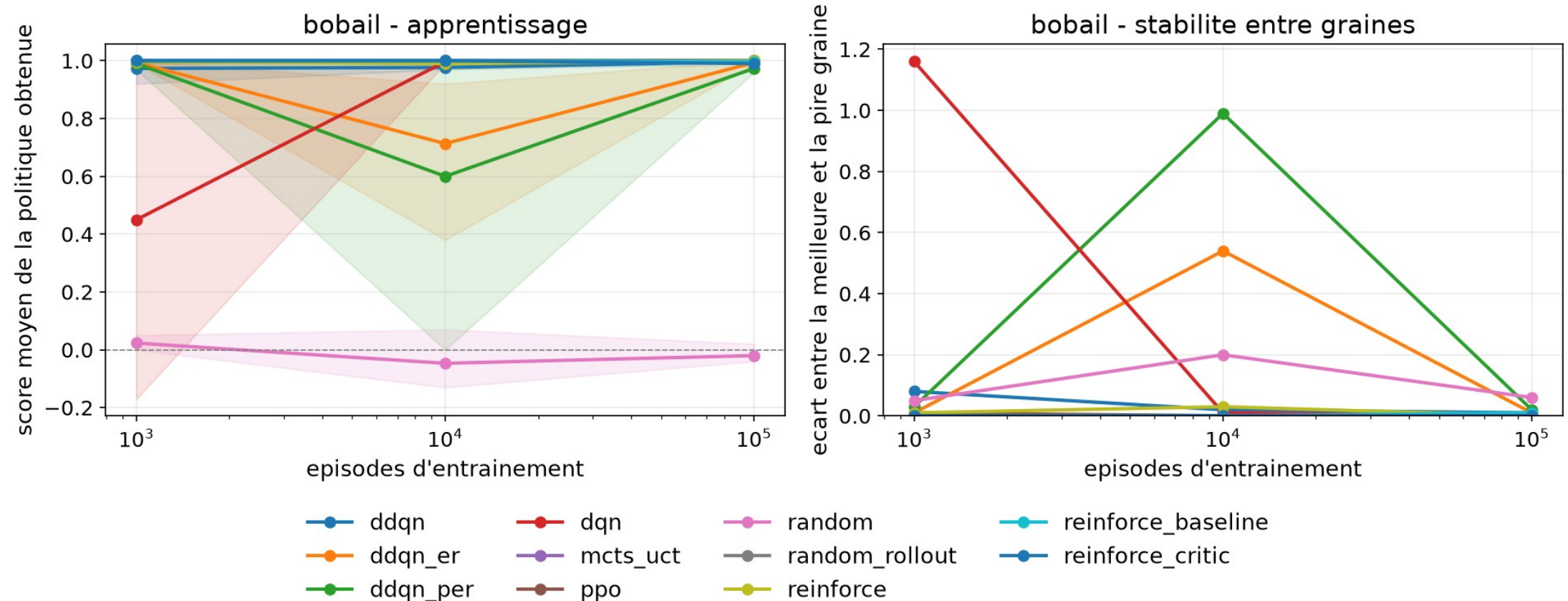
Récapitulatif des agents et résultats

Agent	Objectif et particularité	vs aléatoire	vs heuristique
Random	joue au hasard parmi les coups légaux — la barre à battre	-0,02	-1,00
Deep Q-Learning	apprend une valeur par action, joue l'argmax	+1,00	-0,63
Double Deep Q	ajoute un réseau cible : la cible arrête de bouger	+1,00	+0,80
Double DQN + Replay	rejoue ses anciennes parties : casse la corrélation	+0,99	+0,98
Double DQN + Replay priorisé	rejoue en priorité ce qu'il rate encore	+0,97	+0,97
REINFORCE	apprend la politique directement, en fin de partie	+1,00	+0,57
REINFORCE + baseline	centre les retours : moins de variance	+1,00	+0,09
REINFORCE + critique	remplace la baseline par un réseau $V(s)$	+0,99	+0,86
PPO (A2C style)	borne chaque mise à jour : plusieurs passes sans exploser	+1,00	+1,00
Random Rollout	simule 20 fins de partie par coup, sans apprendre	+1,00	-0,69
MCTS (UCT)	construit un arbre de 200 simulations par coup	+1,00	-0,52

Mesuré jusqu'à 100 000 épisodes, contre deux adversaires. Score de la politique obtenue, moyenne sur 3 graines. Hyperparamètres communs : deux couches de 128 unités, $\gamma = 0,99$.

Courbes d'apprentissage

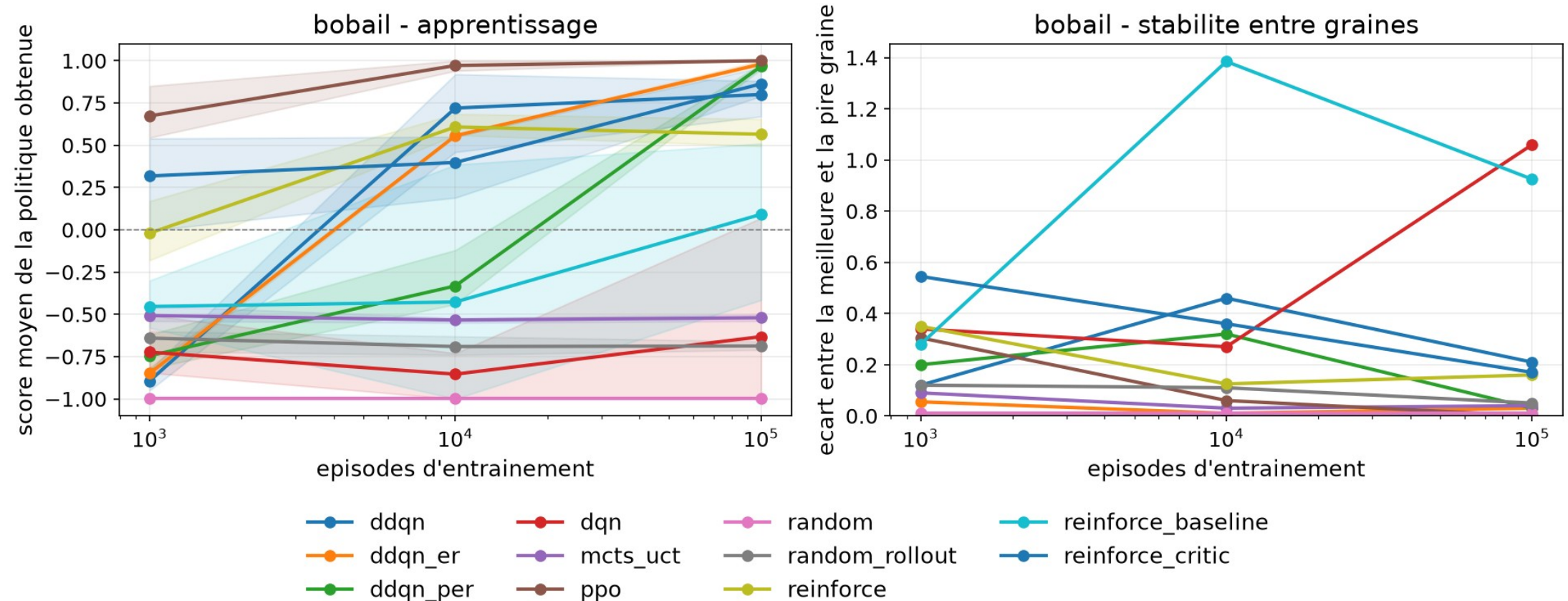
Adversaire aléatoire — tous les agents apprenants convergent vers +1,00



L'environnement ne discrimine pas : c'est ce constat qui a motivé une seconde campagne contre un adversaire compétent.

Courbes d'apprentissage

Adversaire heuristique — les agents s'étalent de $-1,00$ à $+1,00$



Même code, même protocole, mêmes hyperparamètres : seul l'adversaire change.

Ce que cet environnement nous apprend

- **L'adversaire change tout.** Contre l'aléatoire, dix agents sur onze atteignent +1,00 : aucune comparaison possible. Contre l'heuristique, ils s'étalent de -1,00 à +1,00.
- **La planification s'effondre.** MCTS passe de +1,00 à -0,52, Random Rollout à -0,69 — pour 66 ms par coup contre 0,028 ms à PPO, soit 2 400 fois plus cher.
- **PPO gagne, et ne dégrade pas.** +1,00 contre les deux adversaires. Deep Q simple échoue à -0,63 là où ses trois variantes réussissent : c'est la meilleure preuve de l'utilité du réseau cible et du rejeu.

COMMENT L'AMÉLIORER

Porter le palier d'un million, et opposer les agents à un adversaire de force croissante — voire à eux-mêmes en self-play — pour mesurer une généralisation et non une performance contre un adversaire donné.

Ce que l'étude établit

- **Trois environnements sur quatre sont saturés.** Seuls TicTacToe et Bobail contre un adversaire compétent autorisent une comparaison. C'est un résultat sur le protocole autant que sur les algorithmes.
- **Sur une tâche simple, cinq algorithmes convergent au même niveau.** Ce qui les sépare est la fiabilité de la convergence, lisible dans l'écart entre graines à mi-entraînement, et non le score final.
- **La recherche sans apprentissage a une limite mesurable.** MCTS et Random Rollout gagnent contre un adversaire faible, perdent contre un adversaire compétent, et coûtent 2 400 fois plus par décision.
- **Dans la famille gradient de politique, le rapport clippé est décisif.** $0,82 \rightarrow 0,98$ sur TicTacToe, $+0,57 \rightarrow +1,00$ sur Bobail contre l'heuristique.

Ouverture

- **Un adversaire de force croissante.** Nos agents sont mesurés contre un adversaire donné. Les opposer à une série d'adversaires de plus en plus forts mesurerait une généralisation, pas une performance.
- **Le self-play.** Faire jouer un agent contre ses propres versions antérieures supprime la question du choix de l'adversaire.
- **Les algorithmes optionnels du sujet.** Expert Apprentice, AlphaZero et MuZero combinent apprentissage et recherche — exactement l'arbitrage que nos mesures mettent en évidence sans le résoudre.
- **Un environnement volontairement difficile.** Nos trois environnements de test saturent. Un cas plus dur ferait ressortir des écarts que nos mesures ne peuvent pas voir.

Questions

github.com/KaabiOmar/drl-project

165 entraînements • 11 agents • 4 environnements • 30 tests unitaires